Sila API — Mobile Developer Guide

Oct 22, 2026 · @Mohammad

Overview

The mobile app moves from the legacy gateway (`:9201`) to the Sila API. The stored procedures and their operations stay the same; only how you sign in and how you send a call change.

Base URL	`https://sila.glcpaints.com:7400`
Database key	`SILA101` (sent as `database` in every call)
Format	JSON over HTTPS, `Content-Type: application/json`
Auth	`Authorization: Bearer <accessToken>` from sign-in
Health check	`GET /health/live` → `{"status":"alive"}`

What changes from the legacy gateway:

- **You never send the user.** The server takes it from the token. Don't send `User` , `IP` , `SqlStatement` , `State` or `Message` ; the server refuses them.
- **Only allow-listed operations run.** Each procedure + operation pair must be on the server's list (section Mobile operations); anything else returns 403.
- **One endpoint for all procedures**: `POST /api/v2/exec` , with the procedure in the body instead of the `sp_name` header.
- **Login and Change Password** go through `/api/v2/auth/*` , not `APIGeneralOperation` .
- **Live updates** come over a WebSocket on the same server, signed in with a one-time ticket.

Sign in

Sign in once with the user's Sila username and password; you get a 5-minute access token and a 7-day refresh token.

Endpoint	Body	Returns
`POST /api/v2/auth/login`	`{"database":"SILA101","username":"…","password":"…"}`	200 + tokens
`POST /api/v2/auth/refresh`	`{"database":"SILA101","refreshToken":"…"}`	200 + a new pair
`POST /api/v2/auth/logout`	none; Bearer token	204
`POST /api/v2/auth/change-password`	`{"currentPassword":"…","newPassword":"…"}`; Bearer token	200; sign in again

Login and refresh return:

```
{
  "accessToken": "eyJ…",
  "refreshToken": "43 characters",
  "tokenType": "Bearer",
  "expiresAt": "2026-10-22T10:05:00Z",
  "sessionExpiresAt": "2026-10-29T10:00:00Z",
  "database": "SILA101",
  "username": "mhd",
  "displayName": "Mohammad Al Kholy",
  "mustChangePassword": false
}
```

Rules:

- **Access token:** send it as `Authorization: Bearer <accessToken>` on every other call. It expires at `expiresAt` (5 minutes or less); refresh shortly before.
- **Refresh token:** it works once. Each refresh returns a new one; store it and discard the old. Using an old one again ends the whole session (it's treated as stolen). Never send two refreshes at the same time, and never retry a refresh whose reply was lost: sign in again instead.
- **Session:** lasts 7 days from login (`sessionExpiresAt`); refreshing doesn't extend it. Logout, a password change, or the account being disabled ends it at once.
- **Storage:** keep the refresh token in the platform's secure storage (Android Keystore / iOS Keychain), never in plain preferences or logs.
- **Wrong password:** 401 with a generic message. 5 failed attempts within 15 minutes block that user for 15 minutes.

- **First password:** a user who has never changed the initial password can't sign in here yet; they change it in the current app first.
- **New password:** 12 to 100 characters and different from the current one. After a change, every session of the user ends.

Calling operations

Every procedure call is one `POST /api/v2/exec` with the Bearer token; the procedure and operation go in the body.

```
POST https://sila.glcpaints.com:7400/api/v2/exec
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "database": "SILA101",
  "proc": "GRP.APIGroupChatOperation",
  "operation": "Get Left-Right Side Group",
  "lineData": null,
  "extraParams": { "GroupID": 42, "LastChatID": 0, "MinChatID": 0,
"PlatForm": "android" }
}
```

Field	Rules
`database`	Always `SILA101`.
`proc`	`Schema.Name` without brackets, spelled exactly as in the list below (case matters).
`operation`	Exactly as in the list below, including spaces and case (`Left side`, `add admin`).
`lineData`	What the legacy app sent as `LineData`: an object or array (sent to SQL as JSON text), a string or a number (a bare id, e.g. `add admin`, `Forward Ticket`), or `null`.
`extraParams`	Every other procedure parameter, by its name without `@` (`GroupID`, `LastChatID`, `PlatForm`, `FireBaseToken`, …). Only the names listed for that procedure are accepted.

Type rules: numbers as JSON numbers (`42` , not `"42"`); dates as `yyyy-MM-dd` ; date-times ISO 8601. A parameter you leave out keeps its SQL default; an explicit `null` sends SQL NULL.

Never send `User` , `Username` , `IP` , `SqlStatement` , `State` or `Message` : the server fills the user and IP itself and refuses the call (400) if you send them.

A successful call returns 200:

```
{
  "ok": true,
  "state": 0,
  "message": "",
  "data": { "List0": [ { "ChatID": 1001, "Text": "…" } ], "List1": [] },
  "outputs": { "State": 0, "Message": "" },
  "traceId": "80000ce6-0004-cd00-b63f-84710c7967bb"
}
```

`data` holds the procedure's result sets as `List0` , `List1` , … in order, with the same column names as before. When the procedure reports a failure (`@State` not 0, or a single `State` / `Message` row), the reply is 422 with `ok: false` and the message. Quote the `traceId` when reporting a problem.

Live updates (WebSocket)

The socket tells the app *that* something changed and who is online; the app then fetches the change through `/api/v2/exec` . It never carries message text.

1. Get a ticket: `POST /api/v2/live/ticket` with the Bearer token and `{"database":"SILA101"}` → `{"ticket":"…","expiresAt":"…"}` . A ticket works once and expires after 30 seconds.
2. Open `wss://sila.glcpaints.com:7400/api/v2/live?ticket=<ticket>` (URL-encode the ticket). Use a new ticket for every connection.
3. Read JSON text frames until the socket closes, then reconnect with a new ticket.

Frames from the server:

`type`	Fields	Meaning
`hello`	`user` , `online` (usernames), `serverTime`	Sent once after connecting.
`presence`	`user` , `online` (true/false)	Someone came online or went offline.

`type`	Fields	Meaning
`changed`	`seq`, `items`: `[{kind, id, silaId, seq}]`	Conversations of this user changed; refetch them.
`pong`	none	Reply to your `{"type":"ping"}`.

`kind` in `changed` items: `chat` (`id` = SessionID), `group` (GroupID), `ticket` (TicketID), `mail`, `task`, `other`. `silaId` is the Home-list row; `seq` is its new `SynSequence`, so you can pass the highest one as `MaxSeq` to `Left side`.

The only frame you send is `{"type":"ping"}`; others are ignored. The server also sends WebSocket pings every 15 seconds.

Close codes:

Code	Meaning	What to do
1000	Normal close	Reconnect if you still want updates.
1001	Server restarting	Reconnect after a short wait.
1008	Too many frames, or the send failed	Reconnect with backoff; don't flood.
1009	A frame over 4 KB	Fix the client; then reconnect.
4401	Session ended (logout, password change, account disabled)	Refresh the token; if that fails, sign in again.

Reconnect with backoff (2 s, 4 s, 8 s … up to 60 s), and at once when the network returns. After every reconnect, refresh the Home list and the open conversation: events sent while you were offline are not replayed. Each user can have up to 10 sockets open.

Mobile operations

47 operations are open to the app, across four procedures; anything not listed returns 403. Spell `proc` and `operation` exactly as shown.

dbo.APIGeneralOperation

`extraParams` accepted: `SilaID`, `NotificationID`, `LastChatIDOneToOne`, `LastChatIDGroup`, `MaxSeq`, `UnreadList`, `FireBaseToken`, `WebVersion`, `AndroidVersion`, `IOSVersion`, `DeskTopVerion`, `PlatForm`.

Operation	What it does
`Left side`	Home list; send `MaxSeq` for changes since your last load (0 = full list)
`Get Unread Counters`	Unread counts per module
`Get General Notifications`	The user's notifications (marks them seen)
`Delete Notification`	Delete notifications by `NotificationID`
`Hide`	Hide a conversation (`SilaID`)
`UnHide`	Unhide a conversation (`SilaID`)
`Logout`	Remove this device's Firebase token for `PlatForm` (call before `/api/v2/auth/logout`)

Not available yet: `System Users and Groups` (disabled until reviewed). `Login` and `Change Password` are replaced by `/api/v2/auth/*`.

One.APIOneToOneChatOperation

`extraParams` accepted: `SessionID` , `LastChatID` , `MinChatID` , `FireBaseToken` , `AppVersionWeb` , `AppVersionAndroid` , `AppVersionIos` , `AppVersionDesktop` , `PlatForm` .

Operation	What it does
`Get Left-Right Side One-to-one`	Messages of a chat (`LastChatID` / `MinChatID`)
`Load More`	Older messages
`New Chat`	Send a message
`Edit Chat`	Edit a message
`Delete Chat`	Delete a message

GRP.APIGroupChatOperation

`extraParams` accepted: `GroupID` , `LastChatID` , `MinChatID` , `FireBaseToken` , `AppVersionWeb` , `AppVersionAndroid` , `AppVersionIos` , `AppVersionDesktop` , `PlatForm` .

Operation	What it does
`Get Left-Right Side Group`	Messages of a group (`LastChatID` / `MinChatID`)

Operation	What it does
`Load More`	Older messages
`New Chat`	Send a message
`Edit Chat`	Edit a message
`Delete Chat`	Delete a message
`New Group`	Create a group
`Edit Group`	Rename / edit a group
`Get Groups Members`	Members of a group
`add members`	Add members
`add admin`	Make a member admin; `lineData` = the UserID
`remove member`	Remove a member; `lineData` = the UserID
`Left Group`	Leave the group; `lineData` = the UserID
`Lock`	Lock the chat (only admins post)
`UNLock`	Unlock the chat

CHA.APITicketOperation

`extraParams` accepted: `LineMember`, `TicketID`, `TempTicketID`, `ChannelID`, `LastChatID`, `MinChatID`, `MaxSeq`, `FireBaseToken`, `AppVersionWeb`, `AppVersionAndroid`, `AppVersionIos`, `AppVersionDesktop`, `PlatForm`. `LineMember` may be a JSON array; the server converts it to text.

Operation	What it does
`Channel List`	Channels
`Get Left Side Ticket`	Ticket list
`Get Right Side Ticket`	Messages of a ticket
`Get Closed Tickets`	Closed tickets
`New Ticket`	Open a ticket
`New Chat`	Send a ticket message
`Edit Chat`	Edit a ticket message

Operation	What it does
`Delete Chat`	Delete a ticket message
`Start Ticket`	Start
`Hold Ticket`	Hold
`End Ticket`	End
`Close Ticket`	Close
`Cancel Ticket`	Cancel
`Reopen Ticket`	Reopen
`Forward Ticket`	Forward; `lineData` = the target ChannelID
`Accept Forward`	Accept a forward
`Reject Forward`	Reject a forward
`Invite Member`	Invite members (`LineMember`)
`Left Member`	Remove / leave members (`LineMember`)
`Add favorite`	Favorite a channel
`Remove favorite`	Unfavorite a channel

Errors and limits

Every error reply has the same shape, `{"ok":false,"state":1,"message":"…","traceId":"…"}`; branch on the HTTP status, show `message`, and log `traceId`.

Status	Meaning	What the app does
200	Success	Use `data`.
400	Bad request: wrong field, wrong type, a reserved or unlisted parameter	Fix the call; don't retry.
401	Token missing, expired or session ended	Refresh once and retry; if refresh fails, go to sign-in.
403	Operation not allowed, or password change required	Don't retry.

Status	Meaning	What the app does
422	The procedure refused the action (its `State` / `Message`)	Show `message`.
429	Too many requests	Wait, then retry reads.
502	Result too large or invalid	Ask for less (paging).
503	Server not configured for this	Report it with the `traceId`.
504	SQL timeout	Check whether a write went through before trying it again.
500	Other server error	Same as 504 for writes.

Retry rules: retry **reads** (lists, messages) after a short wait. **Never** retry a write (send, edit, delete, state change) automatically after a timeout or a lost reply; it may already have been saved. Refetch the conversation and check first.

Limit	Value
Requests per signed-in user	600 per minute
Sign-in, refresh, change password	10 per minute per IP
Request body	1 MB
SQL time per call	30 s
Result size	25,000 rows / 32 MB per call
Live sockets per user	10
Live frame size / rate	4 KB, 20 frames per 10 s

Quick start with Postman

The Postman collection has the sign-in, refresh, logout, live-ticket and Home-list calls ready to run.

1. In Postman: **Import → Link**, paste `https://raw.githubusercontent.com/malkholy/Sila/main/SilaApi/postman/SilaApi-Mobile.postman_collection.json`.
2. On the collection's **Variables** tab, set `username` and `password` to a test account. Don't save a real password in a shared copy.
3. Run **Login**. It stores `accessToken` and `refreshToken` in the collection variables.

4. Run **Left side (APIGeneralOperation)** to load the Home list. Any other operation: copy that request and change `proc`, `operation` and `extraParams` from the tables above.
5. When calls start returning 401 (after about 5 minutes), run **Refresh**.

More detail: API overview · Integration guide.